ZentroMart VoiceOps Capstone - Final Project Report

Prepared: 25 October 2026 **Dataset reference date:** 20 October 2026 **Project:** Synthetic customer support assistant with text, retrieval, verified tools, and voice adapters

Executive summary

This capstone builds a customer-support assistant in four stages. It begins with a prompt-only baseline using local Qwen3 4B, adds a local embedding vector index over the supplied synthetic company knowledge base, introduces multi-turn order verification and guarded support-ticket tools, then adds Gnani speech-to-text and text-to-speech adapters. The data and policy rules are synthetic and are used only for this educational project.

On the same 52 practice questions and normalized required-fact/safety checker, the prompt-only baseline passed 8/52 while RAG passed 52/52. RAG also found every gold evidence source for 47/47 answerable questions at top five. The final multi-turn result is 96/96: the first full suite passed 92/96, then the four corrected scenarios passed on a 17-turn focused replay. Three user-recorded audio questions were processed sequentially through Gnani STT, the local Qwen support flow, and Gnani TTS. The completed 1:51 demo shows a document-grounded answer, a follow-up, and a human handoff.

Problem and goals

Customers ask policy questions and questions about particular orders. The assistant must answer from company sources, ask for the order ID and verify a second identifier before disclosing order-specific information, avoid exposing personal data, and create a human handover only for qualifying escalations. The final stage must accept audio, transcribe it through Gnani, run the support flow, synthesize an audio reply through Gnani, and save outputs and evidence references for batch recordings.

Implementation

Stage 1 - Prompt-only baseline

The local Qwen3 4B model answers without document retrieval. This stage establishes a simple baseline and shows the limits of relying on the model's general knowledge. Ollama serves the model locally; no hosted LLM API key is required.

Stage 2 - Retrieval-augmented generation

A local `nomic-embed-text` model embeds policy sections and approved catalog/pincode records. A persistent vector cache stores the embeddings outside the submission package. Retrieval fuses cosine similarity with BM25 lexical ranking, exact SKU matching, English/Hindi/Hinglish search variants, and policy routes for safety-critical topics. Retrieved evidence is attached to model context and source IDs are exposed. Business-day refund timelines use the policy's duration and supplied holiday calendar; return deadlines use calendar-day arithmetic and verified order dates.

Stage 3 - Multi-turn tools

The tool agent stores conversation state in memory and supports order lookup and ticket creation via the dataset's local mock tools. A specific order lookup requires an order ID and either the registered phone's last four digits or delivery pincode. Verification proof and contact fields are filtered from model context and ticket summaries. The code limits verification attempts and gates ticket creation on explicit human request or policy escalation conditions. The dataset's tools are mock educational tools, not production services.

Stage 4 - Voice

Gnani STT and TTS are implemented through the official gnani-vachana Python SDK. The batch runner accepts mono 16 kHz, 16-bit PCM WAV files and saves a transcript, assistant text, synthesized WAV, tool-call record, and the retrieved evidence chunks (IDs and text) for each recording. Empty transcripts receive a spoken request to repeat. Three user-recorded clips were processed sequentially with `--conversation`; the demo plays the questions and generated replies, including a human handoff. This is batch audio-file processing rather than continuous live microphone streaming, and spoken order verification is not shown in the video.

Evaluation

Stage	Evaluation	Result	Interpretation
Prompt-only baseline	52 practice questions	8/52 passed	Same normalized required-fact/safety checker; no document evidence; 49/52 escalated conservatively
Retrieval	47 answerable questions, top-k 5	47/47 all gold evidence found	Local embedding + BM25 hybrid index; 100% evidence coverage on the answerable subset
RAG	Same 52 practice questions	52/52 passed	Same normalized required-fact/safety checker; first full run was 48/52, followed by a 4/4 focused recheck; 47/47 answerable questions had all gold sources at top five
Multi-turn tools	96 supplied turns	96/96 passed	Full run 92/96 plus focused replay 17/17 across four corrected scenarios
Gnani voice	Three user-recorded questions; sequential batch	Demonstrated	Official Gnani SDK STT/TTS with local Qwen reply; 1:51 demo shows a policy question, follow-up, and handoff

The practice set is synthetic and small; results do not guarantee performance on unseen evaluation prompts. RAG and multi-turn results include focused replays after the initial full runs and are saved with their methodology in `work/results/`. The 96/96 tool-agent score covers only the supplied synthetic conversation turns.

What went wrong and what I would improve

The prompt-only baseline had no source evidence and passed only 8/52 questions; adding document retrieval raised the same checker result to 52/52. The first full RAG run passed 48/52. Reviewing the misses led to tighter current-policy routing and evidence handling; the four targeted questions then passed 4/4. In the first multi-turn run, 92/96 turns passed. The four misses were two verified-order replies that omitted the promised delivery date, one Hindi coupon reply that did not state the one-coupon limit clearly enough, and one unshipped-order cancellation that was mislabeled as a refusal. The fixes now include the verified delivery date, say that only one coupon applies per order, and distinguish a permitted in-app cancellation from refusing a shipped order. All four affected conversations passed in a 17-turn focused replay.

Next, I would add a live microphone interface, demonstrate spoken order verification, and evaluate more noisy or unclear audio. The blank-transcript recovery now asks the caller to repeat; it still needs broader listening tests. The fact checker uses normalized required phrases, so multilingual paraphrases should also receive human review. Bonus work includes local multilingual embeddings, BM25 and exact-SKU search, policy-priority routes, and code-enforced order-verification and ticket gates.

How to run

Use Python 3.10 or newer, install Ollama and the models `qwen3:4b` and `nomic-embed-text`, and install packages from `requirements.txt`. Extract the supplied dataset to `work/dataset/Capstone_Project_Dataset_FTTT_2026`. Copy `.env.example` to `.env` and set `GNANI_API_KEY` only for voice features. Never publish `.env`.

Commands for the baseline, retrieval, RAG, and tool-agent stages are in the project README. For voice batch processing:

```
python run_voice_batch.py path/to/audio_folder --language en-IN --voice-language auto
```

Voice input must be mono 16 kHz 16-bit PCM WAV. The provided dataset contains no audio recordings.

Submission and limitations

The submission package includes the source, setup guide, report, sample outputs, and a 1:51 demo video. The public GitHub repository provides the source code; the application runs locally and is not deployed as a hosted live demo. The video demonstrates a question answered from store documents, a follow-up, and a human handoff. Continuous live microphone streaming and spoken order verification remain limitations of this demonstration.

Sources

- Supplied *VoiceOps 2026 Capstone Brief* and synthetic FTTT dataset, including its README, policies, practice questions, tables, and mock tools.

- Gnani official SDK, `gnani-vachana`: `https://github.com/Gnani-AI-Mintlify/Gnani-Vachana`; Gnani API documentation at `https://docs.gnani.ai/api/`.

- Ollama local model runtime; Qwen3 4B for generation and `nomic-embed-text` for embeddings.